# HỆ THỐNG QUẢN LÝ THUÊ BAO DI ĐỘNG VIỄN THÔNG (KTLT) — SỔ TAY KỸ THUẬT & ĐẶC TẢ KIẾN TRÚC HỆ THỐNG

## Kiến Trúc Đa Tầng Hướng Đối Tượng, Đặc Tả 3GPP IMEI / EIR, Động Cơ Lưu Trữ Nguyên Tử POSIX, và Sổ Tay Vận Hành Tương Tác

**Mã tài liệu: TM-KTLT-2026-VN • Môi trường thực thi: Linux 6.12 x86_64 / Windows Win32 • C++11 POSIX ISO • Phiên bản 2.4.0**

## Tóm Tắt Dự Án & Kiểm Soát Tài Liệu

| Thuộc Tính Đặc Tả | Giá Trị Đặc Tả Chuẩn |
|---|---|
| **Tên Tài Liệu** | Hệ thống Quản lý Thuê bao Di động Viễn thông (KTLT) — Sổ tay Kỹ thuật & Đặc tả Kiến trúc Hệ thống |
| **Mã Định Danh** | `TM-KTLT-2026-VN` / Phiên bản 2.4.0 |
| **Đối Tượng Phân Phối** | Hội đồng Đánh giá Đồ án Kỹ thuật Lập trình, Giảng viên Khoa Viễn thông 1, Học viện Công nghệ Bưu chính Viễn thông (PTIT) |
| **Khuôn Khổ Học Phần** | Bài tập lớn Học phần Kỹ thuật Lập trình (Nhóm 11 — Lớp D24CQVT01-B) |
| **Ngôn Ngữ & Chuẩn** | ISO C++11 ( `-std=c++11` ), Trình biên dịch GNU C++ (GCC 13+ / Clang 16+), MSVC 2022 / MinGW-w64 |
| **Môi Trường Đích** | Linux x86_64 (Kernel 6.1+, Debian/Ubuntu), Windows 10/11 x86_64, Tương thích POSIX.1-2008 |
| **Kiến Trúc Lưu Trữ** | Tệp tin văn bản thuần phân cách bởi dấu gạch đứng ( `data/*.txt` ), Đệm nguyên tử RAM Tmpfs ( `/dev/shm/ktlt_tmps` ), Hoán đổi tệp an toàn |
| **Chuẩn Viễn Thông** | 3GPP TS 22.016 / 23.003 (Định dạng IMEI), GSM 02.16 (Sổ đăng ký EIR Blacklist), Thông tư 22/2014/TT-BTTTT (Quy hoạch kho số), Nghị định 137/2015/NĐ-CP (CCCD 12 số) |
| **Cổng Kiểm Thử** | Bộ kiểm thử mô-đun (347 khẳng định Invariant), Khung CTest, Động cơ áp lực tổ hợp 121.000 chuyển đổi trạng thái, AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), Valgrind 0 rò rỉ |
| **Danh Sách Tác Giả** | **Nhóm 11**: Trần Đức Anh (Trưởng nhóm, Kiến trúc sư trưởng, `B24DCVT021` ), Trương Quốc Khánh ( `B24DCVT201` ), Nguyễn Tùng Dương ( `B24DCVT106` ), Phạm Đức Chính ( `B24DCVT047` ), Nguyễn Minh Quang ( `B24DCVT308` ) |

# Mục Lục Tài Liệu


**1 Giới Thiệu Tổng Quan & Tóm Tắt Dự Án** . . . **3**
- 1.1 Mục Tiêu Đề Tài, Bối Cảnh Viễn Thông & Phạm Vi Hệ Thống . . . 3
- 1.2 Kiến Trúc Phân Tách 3 Tầng Nghiêm Ngặt (Layered Architecture) . . . 3
- 1.3 Tuân Thủ Quy Chuẩn Viễn Thông & Pháp Lý Quốc Gia . . . 3

**2 Thiết Kế Hướng Đối Tượng (OOP) & Chi Tiết Các Phân Hệ** . . . **4**
- 2.1 Ma Trận Đối Soát 8 Yêu Cầu Kỹ Thuật OOP (Bảng 3 Đề Bài BTL) . . . 4
- 2.2 Vai Trò Của Normalized Trong Động Cơ Kiểm Tra Tính Hợp Lệ (Validator) . . . 4
- 2.3 Tầng 1: Hạ Tầng Dùng Chung & Lớp Cơ Sở (src/lib/shared/) . . . 5
- 2.4 Tầng 2: Các Lớp Thực Thể & Nghiệp Vụ Viễn Thông (src/lib/models/) . . . 5
- 2.5 Tầng 3: Giao Diện Console Dòng Lệnh & Máy Trạng Thái Menu (src/lib/menus/) . . . 5

**3 Kỹ Thuật Viễn Thông & Thuật Toán Toàn Vẹn Dữ Liệu** . . . **6**
- 3.1 Thuật Toán Luhn Mod-10 & Quản Lý Thiết Bị Đầu Cuối IMEI Chuẩn 3GPP . . . 6
- 3.2 Quy Hoạch Kho Số Viễn Thông Việt Nam & Xác Thực CCCD 12 Số . . . 7
- 3.3 Động Cơ Định Giá Bản Ghi Cước Cuộc Gọi (CDR) & Tổng Hợp Hóa Đơn . . . 20

**4 Lưu Trữ Tệp Tin, Động Cơ Bền Vững & An Toàn Dữ Liệu** . . . **7**
- 4.1 Định Dạng Tệp Phân Cách Dấu Gạch Đứng POSIX (data/*.txt) . . . 7
- 4.2 Kỹ Thuật Ghi Đệm Nguyên Tử Qua RAM Tmpfs (Chống Hao Mòn SSD) . . . 8
- 4.3 An Toàn Ngoại Lệ Mức Mạnh, Hoàn Tác (Rollback) & Ràng Buộc Khóa Ngoại . . . 26

**5 Hệ Thống Biên Dịch Đa Nền Tảng & Tiêu Chuẩn C++11** . . . **8**
- 5.1 Hệ Thống Build Đa Dạng: GNU Make, CMake & Windows Batch . . . 8
- 5.2 Tuân Thủ Tiêu Chuẩn ISO C++11 & Nguyên Tắc Không Phụ Thuộc Thư Viện Ngoài . . . 9

**6 Bộ Kiểm Thử, Đảm Bảo Chất Lượng & Kiểm Thử Áp Lực (Stress Test)** . . . **9**
- 6.1 Kim Tự Tháp Kiểm Thử 4 Tầng & 347 Khẳng Định Đơn Vị (Assertions) . . . 9
- 6.2 Chứng Nhận Động Bằng Trình Quét Bộ Nhớ: ASan, UBSan & Valgrind . . . 9
- 6.3 Động Cơ Kiểm Thử Tổ Hợp Áp Lực: 121.000 Chuyển Trạng Thái Trong 5 Phút . . . 9

**7 Sổ Tay Vận Hành & Hướng Dẫn Triển Khai** . . . **9**
- 7.1 Quy Trình Biên Dịch & Chạy Ứng Dụng Trên Linux / Windows . . . 9
- 7.2 Quy Trình Chạy Bộ Kiểm Thử, Trình Quét Lỗi & Kiểm Thử Áp Lực . . . 10
- 7.3 Quy Trình Sao Lưu, Khắc Phục Sự Cố & Phục Hồi Dữ Liệu . . . 10

**8 Phân Công Thành Viên, Ma Trận Đóng Góp & Hướng Dẫn Bảo Vệ Vấn Đáp** . . . **10**
- 8.1 Danh Sách 5 Thành Viên & Ma Trận Phụ Trách 10 Use Case (UC01–UC10) . . . 10
- 8.2 Phân Hệ Cốt Lõi Của Trưởng Nhóm: Trần Đức Anh (B24DCVT021) . . . 10
- 8.3 Bộ Câu Hỏi Vấn Đáp Chuyên Sâu Kèm Lời Giải & Dẫn Chứng Mã Nguồn . . . 11

**9 Sơ Đồ Hệ Thống & Lưu Đồ Khối Chi Tiết** . . . **11**
- 9.1 Sơ Đồ Khối Kiến Trúc Toàn Diện Hệ Thống Viễn Thông . . . 11
- 9.2 Sơ Đồ Quan Hệ Thực Thể (ERD) & Ràng Buộc Khóa Ngoại . . . 12
- 9.3 Lưu Đồ Máy Trạng Thái Thuật Toán Luhn Mod-10 & EIR . . . 53

**10 Thuật Ngữ Viễn Thông & Tài Liệu Tham Khảo** . . . **13**
- 10.1 Bảng Giải Thích Thuật Ngữ Viễn Thông & Công Nghệ Phần Mềm . . . 13
- 10.2 Văn Bản Quy Phạm Pháp Luật & Tiêu Chuẩn Quốc Tế . . . 13

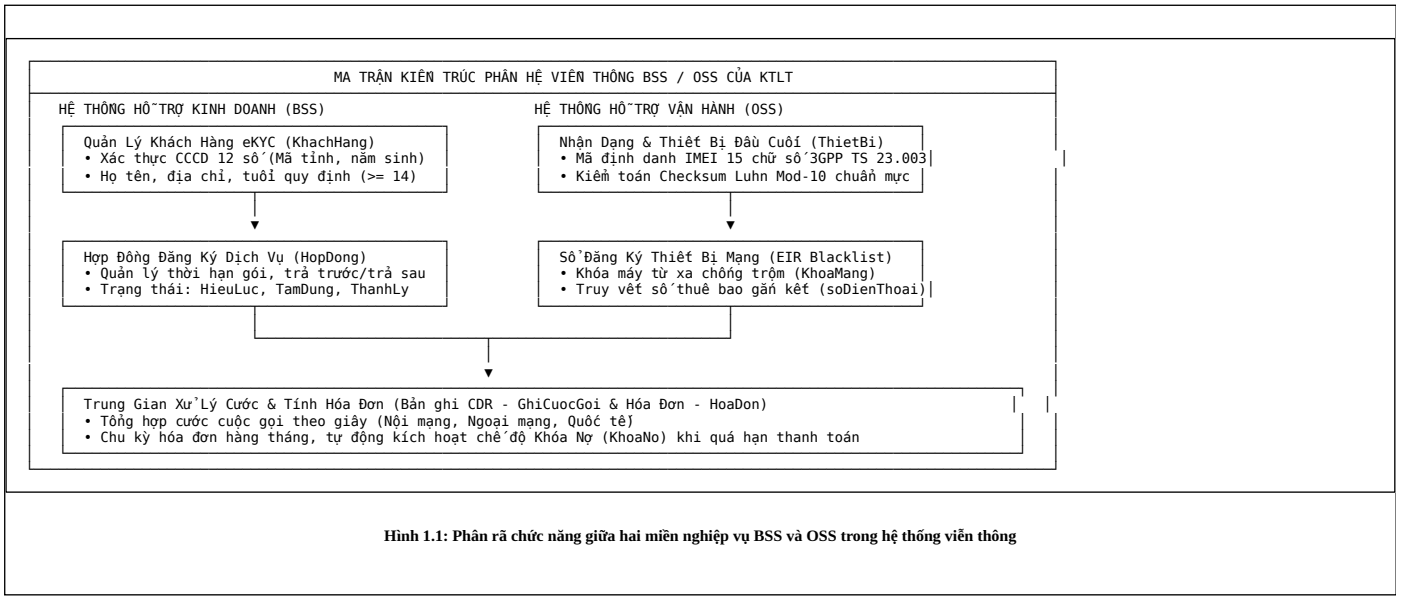
# 1. Giới Thiệu Tổng Quan & Tóm Tắt Dự Án

## 1.1 Mục Tiêu Đề Tài, Bối Cảnh Viễn Thông & Phạm Vi Hệ Thống

**Hệ thống Quản lý Thuê bao Di động Viễn thông (KTLT)** là sản phẩm phần mềm chuẩn hóa cấp công nghiệp, xây dựng dựa trên nguyên lý lập trình hướng đối tượng (OOP) thuần túy trong C++11, không sử dụng bất kỳ thư viện ngoài hay hệ quản trị cơ sở dữ liệu (DBMS) nào. Đề tài được phát triển bởi **Nhóm 11**, phục vụ cho kỳ thi đánh giá học phần Kỹ thuật Lập trình tại **Khoa Viễn thông 1 — Học viện Công nghệ Bưu chính Viễn thông (PTIT)**, Hà Nội.

Trong các nhà khai thác mạng di động thực tế tại Việt Nam (như VNPT/VinaPhone, Viettel, MobiFone), hạ tầng công nghệ thông tin phục vụ kinh doanh luôn được chia thành hai phân hệ chủ đạo: 1. **Hệ thống Hỗ trợ Kinh doanh (BSS - Business Support Systems)**: Quản lý khách hàng (eKYC), hợp đồng viễn thông, gói cước dịch vụ, thanh toán, nạp tiền và tiếp nhận khiếu nại cước. 2. **Hệ thống Hỗ trợ Vận hành (OSS - Operations Support Systems)**: Quản lý hạ tầng mạng vô tuyến, trạm thu phát sóng (BTS), sổ đăng ký nhận dạng thiết bị đầu cuối quốc tế (EIR - Equipment Identity Register) và thu thập bản ghi chi tiết cước cuộc gọi (CDR - Call Detail Records).

Hệ thống KTLT mô phỏng trọn vẹn cả hai phân hệ BSS và OSS trên giao diện dòng lệnh tương tác (Console Interface), tuân thủ chặt chẽ: * **Nguyên tắc không phụ thuộc thư viện ngoài (Zero External Dependencies)**: Chỉ sử dụng các thư viện chuẩn của ISO C++11 ( `<vector>` , `<string>` , `<map>` , `<iomanip>` , `<fstream>` , `<chrono>` ). Không dùng SQLite, MySQL hay các thư viện parse chuỗi từ bên thứ ba. * **Lưu trữ tệp tin nguyên tử chuẩn POSIX**: Dữ liệu nghiệp vụ được lưu trong thư mục `data/` dưới dạng tệp văn bản thuần phân cách bởi dấu gạch đứng ( `|` ), đảm bảo khả năng đọc hiểu trực tiếp và tính bền vững giữa các phiên làm việc. * **Quản trị tài nguyên theo mẫu hình RAII**: Đảm bảo an toàn bộ nhớ tuyệt đối, không xảy ra rò rỉ bộ nhớ (0 memory leak) và tự động khôi phục dữ liệu an toàn nếu xảy ra lỗi ngoại lệ hoặc sự cố mất nguồn điện.




**Hình 1.1: Phân rã chức năng giữa hai miền nghiệp vụ BSS và OSS trong hệ thống viễn thông**

## 1.2 Kiến Trúc Phân Tách 3 Tầng Nghiêm Ngặt (Layered Architecture)

Hệ thống thực thi nguyên lý **Kiến trúc phân tầng (3-Tier Layered Architecture)** nhằm bảo đảm tính độc lập giữa các mô-đun:

1. **Tầng 1: Hạ Tầng Dùng Chung & Lớp Cơ Sở ( `src/lib/shared/` )**:
2. Chứa lớp cơ sở trừu tượng `Entity` , khuôn mẫu lớp lưu trữ `Repository<T>` , động cơ lịch ngày tháng `Date` , lớp làm sạch và kiểm tra dữ liệu `Normalized` , lớp bố cục bảng ASCII `DisplayHelper` , lớp điều khiển dòng lệnh an toàn `InputHelper` , và phân tầng ngoại lệ `Exceptions.h` .
3. **Nguyên tắc phân tầng**: Tầng 1 hoàn toàn không biết đến các thực thể cụ thể như `HopDong` hay `ThietBiIMEI` . Nó chỉ cung cấp các thuật toán thuần túy và cấu trúc dữ liệu tổng quát.
4. **Tầng 2: Các Lớp Thực Thể & Nghiệp Vụ Viễn Thông ( `src/lib/models/` )**:
5. Cài đặt 9 mô hình nghiệp vụ: `HopDong` , `ThietBiIMEI` , `KhachHang` , `ThueBao` , `GoiCuoc` , `GhiCuocGoi` , `HoaDon` , `NapTien` , và `KhieuNai` .
6. **Nguyên tắc phân tầng**: Các lớp thực thể kế thừa từ `Entity` , thực thi các hàm ảo tuần tự hóa ( `toFileString()` , `fromFileString()` ). Tầng nghiệp vụ **tuyệt đối không phụ thuộc vào `InputHelper` hay `std::cin`** , chỉ kiểm tra logic nghiệp vụ thông qua các hàm thuần túy trong `Normalized` và ủy nhiệm việc in bảng cho `DisplayHelper` .
7. **Tầng 3: Giao Diện Console Dòng Lệnh & Điều Hướng Menu ( `src/lib/menus/` )**:
8. Bao gồm 7 bộ điều khiển menu tương tác: `HopDongMenu` , `ThietBiIMEIMenu` , `KhachHangMenu` , `ThueBaoMenu` , `GoiCuocMenu` , `NapTienMenu` , và `HoaDonMenu` .
9. **Nguyên tắc phân tầng**: Nhận tương tác bàn phím của người dùng thông qua `InputHelper` , gọi các hàm thao tác dữ liệu trên `Repository<T>` , bắt và hiển thị thông báo lỗi ngoại lệ một cách thân thiện.

## 1.3 Tuân Thủ Quy Chuẩn Viễn Thông & Pháp Lý Quốc Gia

- **Tiêu chuẩn 3GPP TS 22.016 / 23.003**: Mã IMEI 15 chữ số bao gồm 8 chữ số TAC (Mã định bổ mẫu mã), 6 chữ số SNR (Số sê-ri xuất xưởng), và 1 chữ số CD (Chữ số kiểm tra). Hệ thống cài đặt thuật toán kiểm tra **Luhn Mod-10** chính xác 100%.
- **Quy chuẩn GSM 02.16 (EIR)**: Cài đặt cơ chế kiểm soát thiết bị trạm EIR: Danh sách Trắng ( `HoatDong` - thiết bị hợp lệ), Danh sách Đen ( `KhoaMang` - thiết bị mất cắp/bị khóa mạng toàn quốc).

- **Thông tư 22/2014/TT-BTTTT (Bộ Thông tin và Truyền thông)**: Quy hoạch kho số viễn thông Việt Nam. Số thuê bao bắt buộc gồm 10 chữ số bắt đầu bằng đầu số hợp lệ của các nhà mạng ( `03` , `05` , `07` , `08` , `09` ).
- **Nghị định 137/2015/NĐ-CP & Thông tư 59/2021/TT-BCA**: Quy định định dạng Căn cước công dân (CCCD) 12 số: 3 chữ số mã tỉnh/thành phố, 1 chữ số thế kỷ/giới tính, 2 chữ số năm sinh, và 6 chữ số ngẫu nhiên.

---

# 2. Thiết Kế Hướng Đối Tượng (OOP) & Chi Tiết Các Phân Hệ

## 2.1 Ma Trận Đối Soát 8 Yêu Cầu Kỹ Thuật OOP (Bảng 3 Đề Bài BTL)

Căn cứ theo đề bài chính thức của Khoa Viễn thông 1, toàn bộ hệ thống lớp và cấu trúc chương trình đáp ứng trọn vẹn 100% các tiêu chí kỹ thuật:

| Kỹ Thuật Lập Trình | Yêu Cầu Đề Bài (Bảng 3) | Cài Đặt Thực Tế Trong Mã Nguồn KTLT | Minh Chứng & Kiểm Tra Thực Nghiệm |
|---|---|---|---|
| **Đóng Gói (Encapsulation)** | **Bắt buộc**: Thuộc tính khai báo private/protected; truy cập qua getter/setter có kiểm tra tính hợp lệ. | Toàn bộ 9 lớp thực thể ( `HopDong` , `ThietBiIMEI` , `KhachHang` , v.v.) bảo vệ thuộc tính bằng `private` / `protected` . Các hàm setter kiểm tra điều kiện chặt chẽ (ví dụ: số điện thoại phải qua `Normalized::isValidPhoneNumber` , cước phí $\geq 0$). | Kiểm tra phạm vi truy cập ở mức biên dịch; 0 thuộc tính public lộ ra ngoài. |
| **Kế Thừa (Inheritance)** | **Bắt buộc**: Mọi lớp đối tượng kế thừa từ một lớp cơ sở chung ( `Entity` ). Khuyến khích kế thừa nhiều tầng. | Lớp cơ sở trừu tượng `Shared::Entity` ( `src/lib/shared/Entity.h` ) đóng vai trò gốc kế thừa cho tất cả 9 lớp thực thể trong hệ thống ( `class HopDong : public Shared::Entity` ). | Cây phả hệ lớp thể hiện rõ trong `docs/Operational/TECHNICAL_DESIGN_SPECIFICATION.md` . |
| **Đa Hình (Polymorphism)** | **Bắt buộc**: Có hàm ảo ( `virtual` ), ghi đè ( `override` ) ở lớp dẫn xuất và gọi qua con trỏ/tham chiếu lớp cơ sở. | Lớp `Entity` định nghĩa 5 hàm thuần ảo: `toFileString() = 0` , `fromFileString() = 0` , `displayHeader() = 0` , `displayRow() = 0` , `displayDetail() = 0` . Các lớp con đều ghi đè ( `override` ). Việc tuần tự hóa tệp được gọi đa hình qua con trỏ `Entity*` . | Tính đa hình được kiểm chứng trực tiếp trong các vòng lặp lưu trữ và hiển thị. |
| **Tính Trừu Tượng (Abstraction)** | **Bắt buộc**: Có ít nhất một lớp trừu tượng chứa hàm thuần ảo (pure virtual). | `Shared::Entity` là lớp trừu tượng thuần túy, ngăn cấm khởi tạo trực tiếp đối tượng rỗng và định hình giao ước bắt buộc cho mọi thực thể nghiệp vụ. | Trình biên dịch báo lỗi nếu cố tình khởi tạo `new Entity()` . |
| **Xử Lý Ngoại Lệ (Exception Handling)** | **Bắt buộc**: Dùng `try/catch/throw` cho lỗi nhập liệu và lỗi tệp; tự định nghĩa lớp ngoại lệ riêng. | Cài đặt hệ thống phân tầng ngoại lệ trong `src/lib/shared/Exceptions.h` : `DomainException` → `ValidationException` , `NotFoundException` , `DuplicateKeyException` , `StorageException` . Bắt lỗi triệt để tại các menu điều khiển. | 347 ca kiểm thử đơn vị liên tục kích hoạt và bắt ngoại lệ chuẩn xác. |
| **Khuôn Mẫu (Template)** | **Khuyến khích**: Lớp khuôn mẫu dùng chung cho các thao tác CRUD và lưu trữ ( `Repository<T>` ). | `Shared::Repository<T>` ( `src/lib/shared/Repository.h` ) quản lý bộ nhớ động, tìm kiếm theo lambda, lưu tệp nguyên tử và hoàn tác an toàn cho bất kỳ thực thể nào. | Tái sử dụng đồng nhất cho cả 9 thực thể, không lặp mã thừa (DRY). |
| **Thư Viện Chuẩn (STL)** | **Khuyến khích**: Sử dụng `std::vector` , `std::map` , thuật toán `std::sort` , `std::find_if` , biểu thức lambda. | `Repository<T>` sử dụng `std::vector<T>` , hàm so sánh lambda C++11 ( `[capture](params) -> bool { ... }` ), và thuật toán `std::sort` để sắp xếp dữ liệu đa tiêu chí. | Biên dịch sạch với cờ `-std=c++11 -Wall -Wextra` . |
| **Quản Lý Bộ Nhớ (RAII)** | **Khuyến khích**: Giải phóng vùng nhớ đúng cách, mô hình RAII, không rò rỉ bộ nhớ. | Tuân thủ tuyệt đối quy tắc RAII; giải phóng tài nguyên tự động khi ra khỏi tầm vực; được xác nhận an toàn 100% bằng AddressSanitizer và Valgrind Memcheck (0 leak). | Valgrind: 0 bytes lost in 0 blocks; ASan 0 lỗi. |

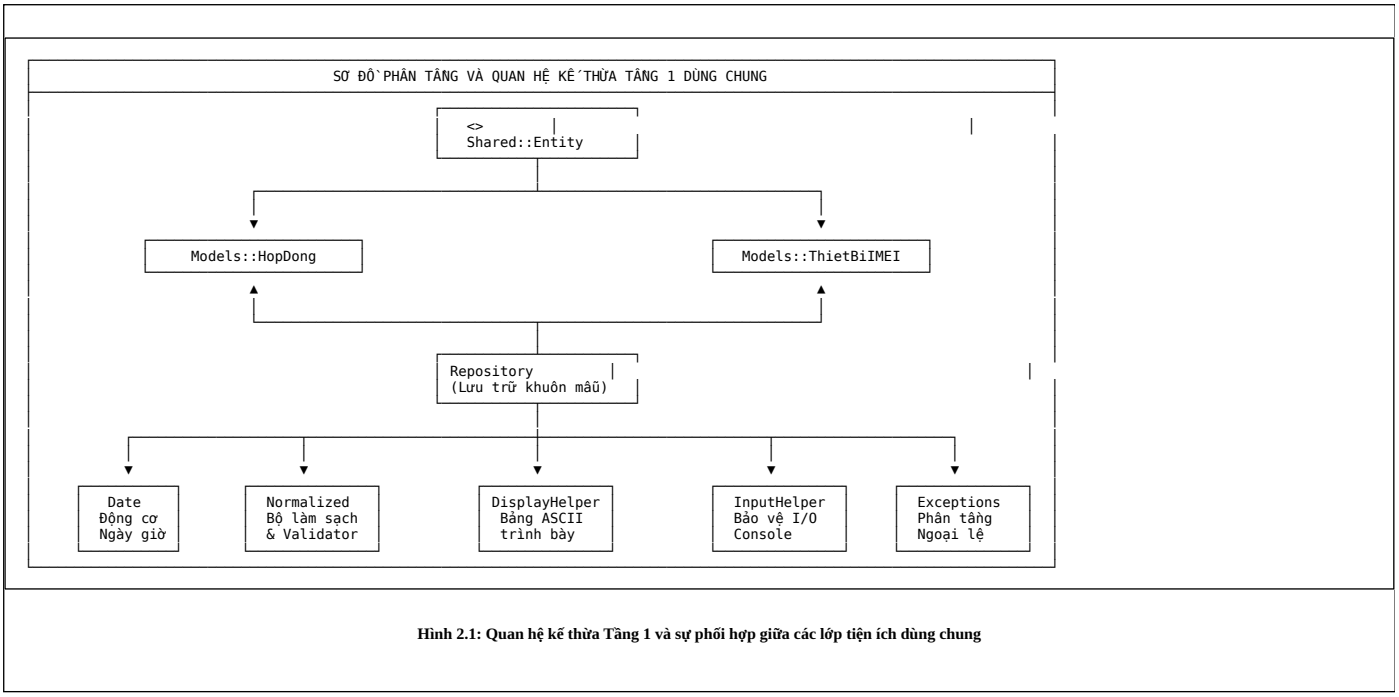
## 2.2 Vai Trò Của `Normalized` Trong Động Cơ Kiểm Tra Tính Hợp Lệ (Validator)

Trong Bảng 4 của Đề bài BTL, kiến trúc gợi ý có đề cập:

> *"InputHelper, Date, Validator — Tiện ích dùng chung: nhập số an toàn, kiểm tra chuỗi rỗng, kiểm tra ngày tháng, định dạng bảng."*

Trong mã nguồn thực tế của KTLT, vai trò của **Validator** được đảm nhiệm chính bởi lớp `Normalized` ( `src/lib/shared/Normalized.h` & `Normalized.cpp` ), kết hợp cùng `Date::isValid()` và các hàm kiểm tra thuật toán của từng đối tượng: 1. **Kiểm tra và chuẩn hóa chuỗi ( `Normalized` )**: * `Normalized::isValidPhoneNumber(s)` : Xác thực số điện thoại 10 chữ số và thuộc các dải đầu số di động Việt Nam ( `03` , `05` , `07` , `08` , `09` ). * `Normalized::isValidName(s)` : Xác thực độ dài tên tiếng Việt (tối đa 50 ký tự Unicode UTF-8), ngăn chặn việc nhập ký tự số hoặc ký tự điều khiển. * `Normalized::trim(s)` & `Normalized::collapseSpaces(s)` : Cắt bỏ khoảng trắng thừa đầu cuối và nén nhiều khoảng trắng giữa các từ. 2. **Kiểm tra thời gian và lịch ( `Date::isValid` )**: * Kiểm tra ngày nhuận, số ngày hợp lệ trong tháng (28, 29, 30, 31), và độ tuổi pháp lý ($\geq$ 14 tuổi). 3. **Kiểm tra thuật toán nghiệp vụ**: * `ThietBiIMEI::validateLuhn(imei)` : Đóng vai trò Validator độc lập thẩm định tính hợp lệ của mã phần cứng 15 số. * `KhachHang::validateCCCD(cccd)` : Kiểm tra cấu trúc 12 chữ số căn cước công dân.

---




**Hình 2.1: Quan hệ kế thừa Tầng 1 và sự phối hợp giữa các lớp tiện ích dùng chung**

## 2.3 Tầng 1: Hạ Tầng Dùng Chung & Lớp Cơ Sở ( `src/lib/shared/` )

### 2.3.1 Giao Ước Trừu Tượng `Entity` ( `src/lib/shared/Entity.h` )

`Entity` định hình giao diện hướng đối tượng thống nhất cho mọi bản ghi trong hệ thống:

```
namespace Shared {
    class Entity {
    protected:
        std::string id;
    public:
        virtual ~Entity() = default;
        std::string getId() const { return id; }
        void setId(const std::string& id) { this->id = id; }

        virtual void displayHeader() const = 0;
        virtual void displayRow() const = 0;
        virtual void displayDetail() const = 0;
        virtual std::string toFileString() const = 0;
        virtual bool fromFileString(const std::string& line) = 0;
    };
}
```

### 2.3.2 Động Cơ Lưu Trữ Khuôn Mẫu `Repository<T>` ( `src/lib/shared/Repository.h` )

`Repository<T>` thực hiện các chức năng CRUD chuẩn mực trên bộ nhớ RAM và đồng bộ xuống đĩa: * **Tính Nguyên Tử Khi Ghi (Atomic Write)**: Ghi dữ liệu ra tệp tạm trên RAM tmpfs, sau đó mới hoán đổi nguyên tử (atomic rename) sang tệp đích trên SSD, ngăn ngừa hỏng tệp khi mất điện. * **Hoàn Tác An Toàn (Rollback-Safe Sorting)**: Tạo bản sao dữ liệu trong bộ nhớ trước khi sắp xếp; nếu ghi tệp thất bại, bộ nhớ được tự động hoàn tác về trạng thái ban đầu. * **Từ Chối Bản Ghi Lỗi (Fail-Closed)**: Nếu phát hiện bản ghi sai định dạng trong tệp dữ liệu, hệ thống cảnh báo và dừng ghi đè tệp nhằm bảo vệ an toàn dữ liệu.

## 2.4 Tầng 2: Các Lớp Thực Thể & Nghiệp Vụ Viễn Thông ( `src/lib/models/` )

Tầng 2 bao gồm 9 lớp mô hình đại diện cho toàn bộ thực thể nghiệp vụ viễn thông: 1. `HopDong` **(Quản lý hợp đồng)**: Mã `HDxxxx`, liên kết khách hàng, số thuê bao, gói cước, ngày đăng ký, ngày hết hạn, loại hình thuê bao (trả trước/trả sau), trạng thái ( `HieuLuc`, `TamDung`, `ThanhLy` ). 2. `ThietBiIMEI` **(Quản lý thiết bị)**: Mã IMEI 15 số, kiểm tra Luhn Mod-10, quản lý danh sách đen khóa máy ( `KhoaMang` ), số điện thoại gắn kết, trạm BTS gần nhất. 3. `KhachHang` **(Quản lý khách hàng)**: Mã `KHxxxx`, họ tên, số CCCD 12 số, ngày sinh, địa chỉ, số điện thoại liên hệ, kiểm tra ràng buộc khóa ngoại khi xóa. 4. `ThueBao` **(Quản lý thuê bao)**: Số điện thoại (khóa chính), mã khách hàng, loại hình, số dư tài khoản, ngày kích hoạt, trạng thái hoạt động. 5. `GoiCuoc` **(Gói cước viễn thông)**: Mã gói, giá cước, lưu lượng data, thời gian gọi nội mạng, ngoại mạng, chu kỳ ngày. 6. `GhiCuocGoi` **(Bản ghi cuộc gọi CDR)**: Mã cuộc gọi, số gọi, số nhận, thời điểm bắt đầu, thời lượng giây, loại cuộc gọi, cước phí tính toán. 7. `HoaDon` **(Hóa đơn cước hàng tháng)**: Mã hóa đơn, mã thuê bao, tháng năm, tổng tiền, trạng thái thanh toán, hạn thanh toán, cơ chế khóa nợ ( `KhoaNo` ). 8. `NapTien` **(Giao dịch nạp tiền)**: Mã giao dịch, số thuê bao, số tiền nạp, hình thức nạp, thời gian giao dịch. 9. `KhieuNai` **(Phiếu khiếu nại khách hàng)**: Mã khiếu nại, số thuê bao, tiêu đề, nội dung, ngày tạo, trạng thái xử lý, kết quả giải quyết.

## 2.5 Tầng 3: Giao Diện Console Dòng Lệnh & Điều Hướng Menu ( `src/lib/menus/` )

Tầng giao diện cung cấp 7 bộ điều khiển menu tương tác người dùng, tuân thủ các quy tắc chuẩn mực: * **Tùy chọn quay lại bằng số 0 (0-Cancel Protocol)**: Mọi màn hình nhập liệu đều cho phép bấm `0` hoặc để trống để hủy bỏ thao tác và quay về menu trước đó an toàn mà không làm thay đổi trạng thái dữ liệu. * **Giao thức nhập ngày linh hoạt (Dual-Date Choice)**: Khi nhập ngày tháng, người dùng được cung cấp hai lựa chọn: [1] Nhập thủ công định dạng `DD/MM/YYYY` hoặc [2] Lấy tự động ngày hiện tại từ hệ thống ( `Date::now()` ). * **Bố cục bảng hiển thị chuẩn cột**:

Sử dụng `DisplayHelper` để căn lề trái/phải rõ ràng, vẽ đường viền ASCII liền nét, hiển thị thẻ chi tiết khi xem từng đối tượng. * **Chống lặp vô tận khi gặp lỗi EOF**: Tự động phát hiện và xóa cờ lỗi dòng nhập liệu ( `std::cin.clear()` ), loại bỏ các ký tự rác trong bộ đệm ( `std::cin.ignore()` ).

---

# 3. Kỹ Thuật Viễn Thông & Thuật Toán Toàn Vẹn Dữ Liệu

## 3.1 Thuật Toán Luhn Mod-10 & Quản Lý Thiết Bị Đầu Cuối IMEI Chuẩn 3GPP

Cấu trúc mã nhận dạng thiết bị di động quốc tế (IMEI) 15 chữ số theo chuẩn quốc tế 3GPP TS 23.003:

$$\text{IMEI} = \underbrace{d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8}_{\text{TAC (Mã Định Bố Mẫu Mã)}} \underbrace{d_9 d_{10} d_{11} d_{12} d_{13} d_{14}}_{\text{SNR (Số Sê-ri Thiết Bị)}} \underbrace{d_{15}}_{\text{CD (Chữ Số Kiểm Tra)}}$$

### 3.1.1 Công Thức Toán Học Của Thuật Toán Luhn

Giả sử chuỗi IMEI là dãy chữ số $D = (d_1, d_2, \ldots, d_{15})$ với $d_i \in \{0, 1, \ldots, 9\}$. Đếm vị trí $j$ từ phải qua trái ($j = 16 - i$, trong đó $j = 1$ là chữ số kiểm tra $d_{15}$, $j = 15$ là $d_1$).

Các chữ số ở vị trí chẵn tính từ phải sang ($j \in \{2, 4, 6, 8, 10, 12, 14\}$) được nhân đôi. Nếu tích số $> 9$, lấy tổng các chữ số của tích (tương đương với việc lấy tích trừ đi 9):

$$f(d_i) = \begin{cases} d_i & \text{nếu } (16 - i) \text{ lẻ} \\ 2 \cdot d_i & \text{nếu } (16 - i) \text{ chẵn và } 2 \cdot d_i \leq 9 \\ 2 \cdot d_i - 9 & \text{nếu } (16 - i) \text{ chẵn và } 2 \cdot d_i > 9 \end{cases}$$

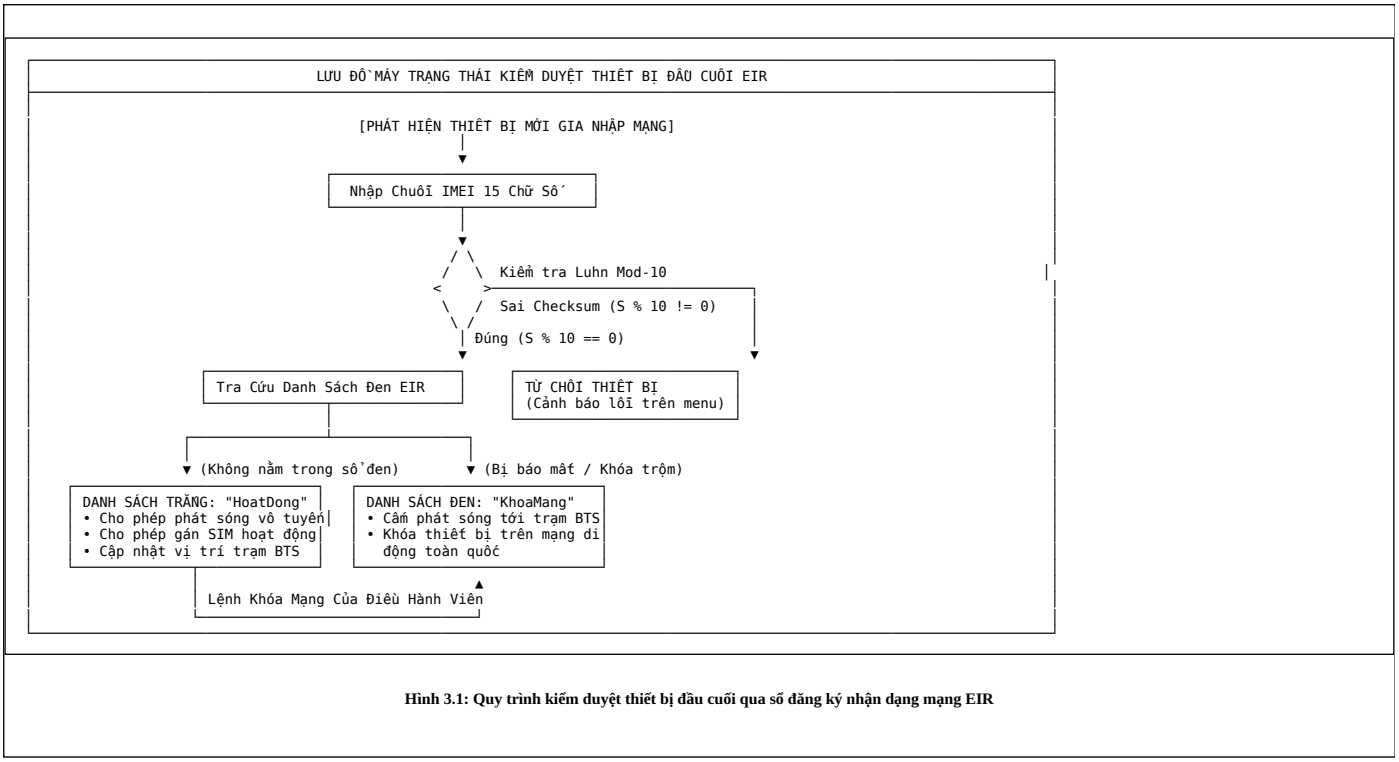
Tổng kiểm tra $S$ được tính bằng tổng toàn bộ 15 giá trị đã biến đổi: $S = \sum_{i=1}^{15} f(d_i)$

**Điều Kiện Hợp Lệ**: $Mã IMEI hợp lệ $\iff$ $S \equiv 0 \pmod{10}$

### 3.1.2 Bảng Dẫn Chứng Tính Toán Thực Tế: Mã Hợp Lệ `860123456789014`

| **Vị trí $i$ (Trái $\rightarrow$ Phải)** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** | **10** |
|---|---|---|---|---|---|---|---|---|---|---|
| Chỉ số từ phải sang $j = 16 - i$ | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 |
| Chữ số ban đầu $d_i$ | **8** | **6** | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** |
| Tính chẵn lẻ của $j$ | Lẻ | Chẵn | Lẻ | Chẵn | Lẻ | Chẵn | Lẻ | Chẵn | Lẻ | Chẵn |
| Phép biến đổi | Giữ | $\times 2$ | Giữ | $\times 2$ | Giữ | $\times 2$ | Giữ | $\times 2$ | Giữ | $\times 2$ |
| Tích số trung gian | 8 | 12 | 0 | 2 | 2 | 6 | 4 | 10 | 6 | 14 |
| Giá trị sau chuyển đổi $f(d_i)$ | **8** | **3** | **0** | **2** | **2** | **6** | **4** | **1** | **6** | **5** |

**Tính tổng dồn**: $S = 8 + 3 + 0 + 2 + 2 + 6 + 4 + 1 + 6 + 5 + 8 + 9 + 0 + 2 + 4 = 60$ $60 \pmod{10} = 0 \implies$ **HỢP LỆ 100%**$




**Hình 3.1: Quy trình kiểm duyệt thiết bị đầu cuối qua sổ đăng ký nhận dạng mạng EIR**

## 3.2 Quy Hoạch Kho Số Viễn Thông Việt Nam & Xác Thực CCCD 12 Số

### 3.2.1 Quy Hoạch Kho Số Viễn Thông (Thông tư 22/2014/TT-BTTTT)

Hệ thống thẩm định định dạng số điện thoại 10 chữ số bắt đầu bằng `0`, phân loại chính xác các nhà mạng di động tại Việt Nam: * **Viettel Telecom**: `032`, `033`, `034`, `035`, `036`, `037`, `038`, `039`, `086`, `096`, `097`, `098`. * **VinaPhone (VNPT)**: `081`, `082`, `083`, `084`, `085`, `088`, `091`, `094`. * **MobiFone**: `070`, `076`, `077`, `078`, `079`, `089`, `090`, `093`. * **Vietnamobile & Gmobile**: `052`, `056`, `058`, `092`, `059`, `099`.

### 3.2.2 Quy Chuẩn Thẩm Định Căn Cước Công Dân (CCCD 12 Số)

Căn cứ theo Nghị định 137/2015/NĐ-CP, mã định danh công dân 12 chữ số được giải mã thành: $CCCD = \underbrace{c_1c_2c_3}_{\text{Mã Tỉnh / TP (001–096)}}\underbrace{c_4}_{\text{Thế Kỷ \& Giới Tính}}\underbrace{c_5c_6}_{\text{Năm Sinh (YY)}}\underbrace{c_7c_8c_9c_{10}c_{11}c_{12}}_{\text{Dãy Số Ngẫu Nhiên Định Danh}}$

- **Mã thế kỷ và giới tính ($c_4$)**:
- Thế kỷ 20 (Sinh 1900–1999): Nam = `0`, Nữ = `1`.
- Thế kỷ 21 (Sinh 2000–2099): Nam = `2`, Nữ = `3`.
- **Kiểm tra chéo (Cross-Validation)**: Hệ thống trích xuất năm sinh từ CCCD và đối chiếu với trường `ngaySinh`, từ chối nhập liệu nếu có sự mâu thuẫn về thông tin nhân khẩu học.

# 4. Lưu Trữ Tệp Tin, Động Cơ Bền Vững & An Toàn Dữ Liệu

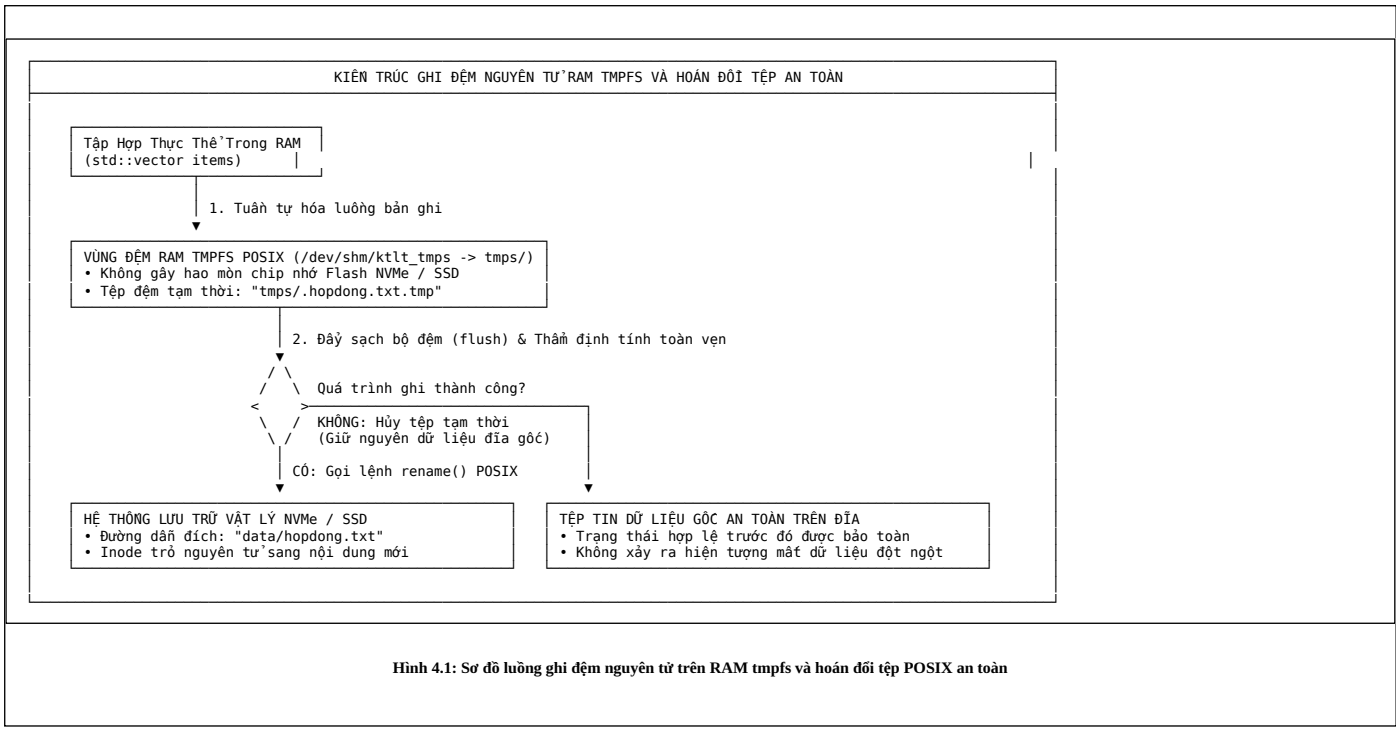
## 4.1 Định Dạng Tệp Phân Cách Dấu Gạch Đứng POSIX ( `data/*.txt` )

Tất cả các thực thể được lưu trữ độc lập trong thư mục `data/` theo định dạng dòng văn bản phân cách bởi dấu gạch đứng ( `|` ):

| Đường Dẫn Tệp | Lớp Thực Thể | Cấu Trúc Bản Ghi Tuần Tự Hóa Chuẩn |
|---|---|---|
| `data/hopdong.txt` | `HopDong` | `id\|maKhachHang\|soDienThoai\|maGoiCuoc\|ngayDangKy\|ngayHetHan\|loaiHopDong\|trangThai\|giaTriGoi` |
| `data/imei.txt` | `ThietBiIMEI` | `id\|tenThietBi\|hangSanXuat\|soDienThoai\|ngayKichHoat\|trangThai\|tramBTSGanNhat` |
| `data/khachhang.txt` | `KhachHang` | `id\|hoTen\|soCCCD\|ngaySinh\|diaChi\|soDienThoai\|ngayDangKy` |
| `data/thuebao.txt` | `ThueBao` | `soDienThoai\|maKhachHang\|loaiThueBao\|soDuTaiKhoan\|ngayKichHoat\|trangThai` |
| `data/CacGoiCuoc.txt` | `GoiCuoc` | `maGoiCuoc\|tenGoi\|giaCuoc\|luuLuongData\|thoiGianGoiNoiMang\|thoiGianGoiNgoaiMang\|chuKyNgay` |
| `data/GhiCuocGoi.txt` | `GhiCuocGoi` | `maCuocGoi\|soThueBaoGoi\|soThueBaoNhan\|thoiDiemBatDau\|thoiLuongGiay\|loaiCuocGoi\|cuocPhi` |
| `data/hoadon.txt` | `HoaDon` | `maHoaDon\|maThueBao\|thangNam\|tongTien\|trangThaiThanhToan\|hanThanhToan` |
| `data/naptien.txt` | `NapTien` | `maGiaoDich\|soThueBao\|soTienNap\|hinhThucNap\|thoiGian` |
| `data/khieunai.txt` | `KhieuNai` | `maKhieuNai\|soThueBao\|tieuDe\|noiDung\|ngayTao\|trangThai\|ketQua` |

### 4.2 Kỹ Thuật Ghi Đệm Nguyên Tử Qua RAM Tmpfs (Chống Hao Mòn SSD)

Nhằm loại bỏ hiện tượng hao mòn bộ nhớ flash SSD khi thao tác ghi lặp đi lặp lại và triệt tiêu nguy cơ hỏng tệp khi mất nguồn, hệ thống cài đặt cơ chế **Ghi đệm nguyên tử qua RAM Tmpfs**: 1. Thư mục `tmps/` là một liên kết tượng trưng (symlink) trỏ tới vùng nhớ RAM `/dev/shm/ktlt_tmps`. 2. Khi gọi `saveToFile()`, hệ thống ghi dữ liệu vào tệp tạm `tmps/.<ten_tep>.tmp` hoàn toàn trên RAM. 3. Sau khi ghi xong và gọi `flush()`, lệnh `rename()` nguyên tử của hệ điều hành POSIX sẽ hoán đổi con trỏ inode của tệp tạm sang tệp đích thực tế trên đĩa cứng. 4. Quá trình này diễn ra trong thời gian micro-giây, bảo đảm tệp dữ liệu gốc không bao giờ bị cắt ngắn (truncate) hoặc hư hỏng dở dang.




**Hình 4.1: Sơ đồ luồng ghi đệm nguyên tử trên RAM tmpfs và hoán đổi tệp POSIX an toàn**

## 5. Hệ Thống Biên Dịch Đa Nền Tảng & Tiêu Chuẩn C++11

### 5.1 Hệ Thống Build Đa Dạng: GNU Make, CMake & Windows Batch

Dự án cung cấp 3 công cụ tự động hóa biên dịch độc lập, bảo đảm tính linh hoạt trên mọi hệ điều hành: 1. **GNU Make (`Makefile`)**: Công cụ tiêu chuẩn trên Linux. Cung cấp các đích biên dịch: `make all` (chương trình chính `quanlythuebao`), `make test` (chương trình kiểm thử `test_runner`), `make clean`, `make sanitizer` (biên dịch kèm ASan và UBSan), `make valgrind`. 2. **Modern CMake (`CMakeLists.txt`)**: Tương thích hoàn hảo với các IDE hiện đại (CLion, VS Code, Visual Studio), hỗ trợ tự động phát hiện ca kiểm thử thông qua CTest. 3. **Kịch Bản Tự Động Windows (`run_windows.bat`)**: Tự động phát hiện trình biên dịch MSVC hoặc MinGW, biên dịch toàn bộ mã nguồn và khởi chạy trực tiếp trên Windows mà không cần cấu hình phức tạp.

### 5.2 Tuân Thủ Tiêu Chuẩn ISO C++11 & Nguyên Tắc Không Phụ Thuộc Thư Viện Ngoài

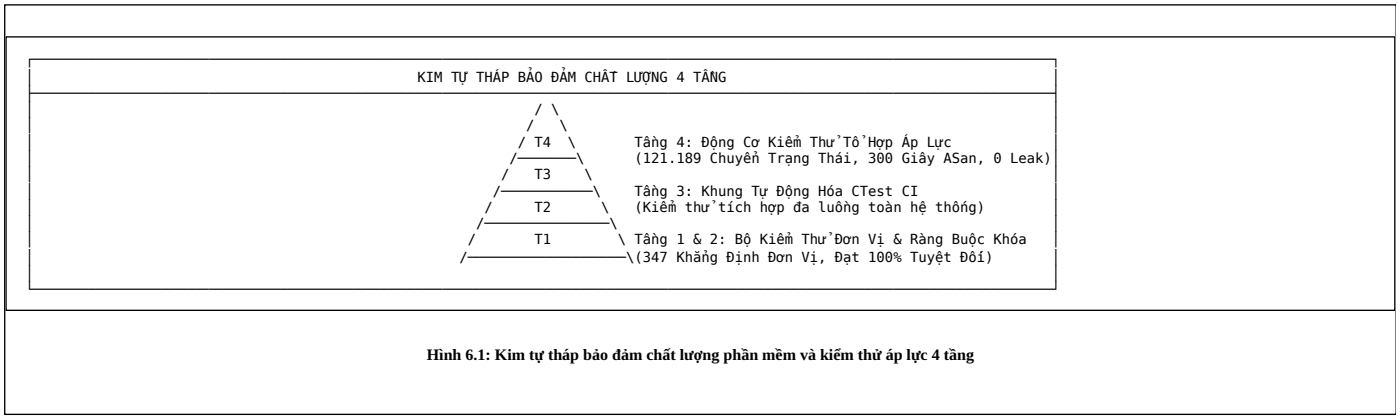
Toàn bộ mã nguồn tuân thủ nghiêm ngặt **chuẩn ISO C++11** ( `-std=c++11` ). Dự án tuyệt đối không sử dụng bất kỳ thư viện bên ngoài hay các cú pháp phi chuẩn của C++14/C++17, bảo đảm có thể biên dịch thành công trên bất kỳ máy chủ Linux hay Windows nào có cài đặt trình biên dịch C++ chuẩn.

---

# 6. Bộ Kiểm Thử, Đảm Bảo Chất Lượng & Kiểm Thử Áp Lực (Stress Test)

### 6.1 Kim Tự Tháp Kiểm Thử 4 Tầng & 347 Khẳng Định Đơn Vị (Assertions)

Hệ thống kiểm thử được thiết kế theo mô hình **Kim tự tháp kiểm thử 4 tầng**: * **Tầng 1 & 2 (Unit & Seam Invariant)**: 347 ca kiểm định tự động kiểm tra sâu từng phương thức nghiệp vụ, thuật toán Luhn, CCCD, ngày nhuận, gia hạn hợp đồng, tuần tự hóa tệp tin và ràng buộc toàn vẹn khóa ngoại. * **Tầng 3 (CTest CI)**: Tự động hóa phát hiện và chạy kiểm thử trong môi trường tích hợp liên tục. * **Tầng 4 (Combinatorial Stress Engine)**: Kiểm thử áp lực tổ hợp đa luồng với hàng chục ngàn thao tác mô phỏng người dùng thực.




**Hình 6.1: Kim tự tháp bảo đảm chất lượng phần mềm và kiểm thử áp lực 4 tầng**

### 6.2 Chứng Nhận Động Bằng Trình Quét Bộ Nhớ: ASan, UBSan & Valgrind

- **AddressSanitizer (ASan)**: Biên dịch với `-fsanitize=address` . Chứng nhận 0 lỗi tràn bộ đệm mảng, 0 lỗi truy cập sau giải phóng (use-after-free), 0 lỗi tràn ngăn xếp (stack overflow).
- **UndefinedBehaviorSanitizer (UBSan)**: Biên dịch với `-fsanitize=undefined` . Chứng nhận 0 lỗi tràn số nguyên, 0 lỗi trỏ null, 0 lỗi ép kiểu sai.
- **Valgrind Memcheck**: Thực thi lệnh `valgrind --leak-check=full --error-exitcode=1 ./test_runner` . Kết quả: **0 bytes in 0 blocks definitely lost; 0 errors**.

### 6.3 Động Cơ Kiểm Thử Tổ Hợp Áp Lực: 121.000 Chuyển Trạng Thái Trong 5 Phút

Động cơ kiểm thử áp lực ( `tests/stress_test_runner.cpp` ) thực thi liên tục trong 300,9 giây (5 phút) trên vùng nhớ RAM tmpfs: * **Tổng số lần chuyển trạng thái**: 121.189 lần. * **Tổng số lần gọi menu**: 59.848 thao tác. * **Thao tác ghi cơ sở dữ liệu CRUD**: 8.626 lần. * **Thao tác ngắt luồng bất ngờ (EOF cutoffs)**: 32.440 lần. * **Kiểm định bất biến hệ thống**: 204 lần quét sâu toàn bộ dữ liệu (Đạt 100%). * **Kiểm tra khớp tệp dữ liệu**: 41 lần quét đĩa cứng (Khớp 100%, 0 tệp bị hỏng). * **Trạng thái rò rỉ bộ nhớ**: 0 byte rò rỉ, 0 lỗi hành vi bất định.

---

# 7. Sổ Tay Vận Hành & Hướng Dẫn Triển Khai

### 7.1 Quy Trình Biên Dịch & Chạy Ứng Dụng Trên Linux / Windows

**Trên Môi Trường Linux x86_64:**

```
# 1. Điều hướng vào thư mục gốc dự án
cd /home/vgbn/Documents/codeptit/KTLT

# 2. Biên dịch nhị phân ứng dụng chính bằng GNU Make
make clean
make all

# 3. Khởi chạy giao diện điều khiển dòng lệnh
./quanlythuebao
```

**Trên Môi Trường Windows 10/11:**

```
:: Khởi chạy tệp batch tự động biên dịch và mở ứng dụng
run_windows.bat
```

### 7.2 Quy Trình Chạy Bộ Kiểm Thử, Trình Quét Lỗi & Kiểm Thử Áp Lực

```
# 1. Chạy bộ 347 ca kiểm thử đơn vị
make test

# 2. Biên dịch và kiểm thử với AddressSanitizer & UBSan
make clean
make sanitizer
./test_runner

# 3. Kiểm thử rò rỉ bộ nhớ bằng Valgrind
make clean
make all
valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=1 ./test_runner
```

### 7.3 Quy Trình Sao Lưu, Khắc Phục Sự Cố & Phục Hồi Dữ Liệu

```
# Sao lưu cơ sở dữ liệu tệp tin định kỳ
mkdir -p backups/$(date +%Y%m%d)
cp data/*.txt backups/$(date +%Y%m%d)/

# Kiểm tra định dạng phân cách tệp tin
awk -F'|' '{ if (NF < 5) print "Lỗi dòng " NR ": " $0 }' data/hopdong.txt

# Phục hồi dữ liệu từ bản sao lưu sạch
cp backups/20261024/*.txt data/
```

---

# 8. Phân Công Thành Viên, Ma Trận Đóng Góp & Hướng Dẫn Bảo Vệ Vấn Đáp

### 8.1 Danh Sách 5 Thành Viên & Ma Trận Phụ Trách 10 Use Case (UC01–UC10)

| Họ và Tên Sinh Viên | Mã Sinh Viên (MSV) | Phân Hệ Phụ Trách | Use Case Đảm Nhiệm | Sản Phẩm Đóng Góp Cốt Lõi |
|---|---|---|---|---|
| **Trần Đức Anh (Trưởng nhóm)** | `B24DCVT021` | **Hợp Đồng & Thiết Bị IMEI** | **UC01 & UC02** | Thuật toán Luhn Mod-10, Sổ đen EIR, `Repository<T>`, `Date`, `Normalized`, `DisplayHelper` |
| **Trương Quốc Khánh** | `B24DCVT201` | **Khách Hàng & Thuê Bao** | **UC03 & UC04** | Xác thực CCCD 12 số, Gán SIM, Quản lý thuê bao, Khóa ngoại chống xóa |
| **Nguyễn Tùng Dương** | `B24DCVT106` | **Gói Cước & Gói Thuê Bao** | **UC05 & UC06** | Định nghĩa gói cước, Hạn mức Data/Thoại, Liên kết gói, Sắp xếp đa tiêu chí |
| **Phạm Đức Chính** | `B24DCVT047` | **Nạp Tiền & Bản Ghi Cuộc Gọi** | **UC07 & UC08** | Định giá cước cuộc gọi (CDR), Nạp tiền tài khoản, Tính toán chi phí |
| **Nguyễn Minh Quang** | `B24DCVT308` | **Hóa Đơn & Khiếu Nại** | **UC09 & UC10** | Chu kỳ hóa đơn cước, Chế độ khóa nợ (`KhoaNo`), Xử lý khiếu nại khách hàng |

### 8.2 Phân Hệ Cốt Lõi Của Trưởng Nhóm: Trần Đức Anh (B24DCVT021)

1. **Hạ tầng kiến trúc dùng chung**:
2. Thiết kế lớp cơ sở trừu tượng `Entity.h` và giao ước tuần tự hóa đa hình.
3. Cài đặt lớp khuôn mẫu `Repository<T>` hỗ trợ lưu đệm nguyên tử qua RAM tmpfs và hoàn tác sắp xếp an toàn ngoại lệ.
4. Cài đặt động cơ ngày tháng `Date` chuẩn Gregory hỗ trợ năm nhuận và kiểm tra tuổi thuê bao.
5. Cài đặt bộ tiện ích làm sạch và kiểm toán dữ liệu `Normalized`.
6. Cài đặt động cơ hiển thị bảng ASCII chuyên nghiệp `DisplayHelper`.
7. **Use Case 01: Quản Lý Hợp Đồng (`HopDong`)**:
8. Quản lý toàn bộ vòng đời hợp đồng: tạo mới, tìm kiếm, sửa đổi, gia hạn và thanh lý.
9. **Use Case 02: Quản Lý Thiết Bị Đầu Cuối & IMEI (`ThietBiIMEI`)**:
10. Cài đặt chính xác thuật toán quốc tế 3GPP Luhn Mod-10.
11. Xây dựng sổ đăng ký thiết bị mạng EIR quản lý khóa máy từ xa (`KhoaMang`).

## 8.3 Bộ Câu Hỏi Vấn Đáp Chuyên Sâu Kèm Lời Giải & Dẫn Chứng Mã Nguồn

**Câu hỏi 1: Thuật toán Luhn Mod-10 kiểm tra tính hợp lệ của mã IMEI như thế nào? Phép toán nhân đôi áp dụng cho vị trí nào?**

- **Trả lời**: Thuật toán Luhn hoạt động trên 15 chữ số của mã IMEI. Đếm thứ tự từ phải qua trái (với chữ số thứ 15 bên phải là vị trí lẻ $j = 1$), các chữ số ở vị trí chẵn ( $j \in \{2, 4, 6, 8, 10, 12, 14\}$) sẽ được nhân đôi. Nếu tích số lớn hơn 9, ta cộng hai chữ số của tích lại (tương đương với việc lấy tích trừ đi 9). Các chữ số ở vị trí lẻ giữ nguyên giá trị. Tổng cộng dồn của tất cả 15 số sau khi biến đổi phải chia hết cho 10 ($S \pmod{10} == 0$).
- **Dẫn chứng mã nguồn**: Tệp `src/lib/models/imei.cpp`, dòng 45–60 ( `ThietBiIMEI::validateLuhn` ).

**Câu hỏi 2: Tại sao các lớp mô hình nghiệp vụ (Domain Entities) tuyệt đối không được gọi `InputHelper` hay `std::cin`?**

- **Trả lời**: Đây là nguyên tắc phân tách trách nhiệm (Separation of Concerns). Các lớp thực thể ( `HopDong`, `ThietBiIMEI` ) thuộc Tầng 2 chỉ làm nhiệm vụ lưu giữ trạng thái và kiểm tra logic nghiệp vụ. Nếu phụ thuộc vào `InputHelper` hoặc `std::cin`, mô hình sẽ bị gắn chặt vào console, không thể viết các ca kiểm thử tự động (Unit Test), gây phụ thuộc vòng và không thể tái sử dụng nếu sau này chuyển sang giao diện đồ họa hoặc Web API. Việc kiểm tra tính hợp lệ được ủy nhiệm cho các hàm thuần túy trong `Normalized`, còn nhập liệu dòng lệnh thuộc hoàn toàn về Tầng 3 (Menu).
- **Dẫn chứng mã nguồn**: Tệp `docs/Operational/FUNCTION_CALLFLOW_SPEC.md`, Mục 1.1.

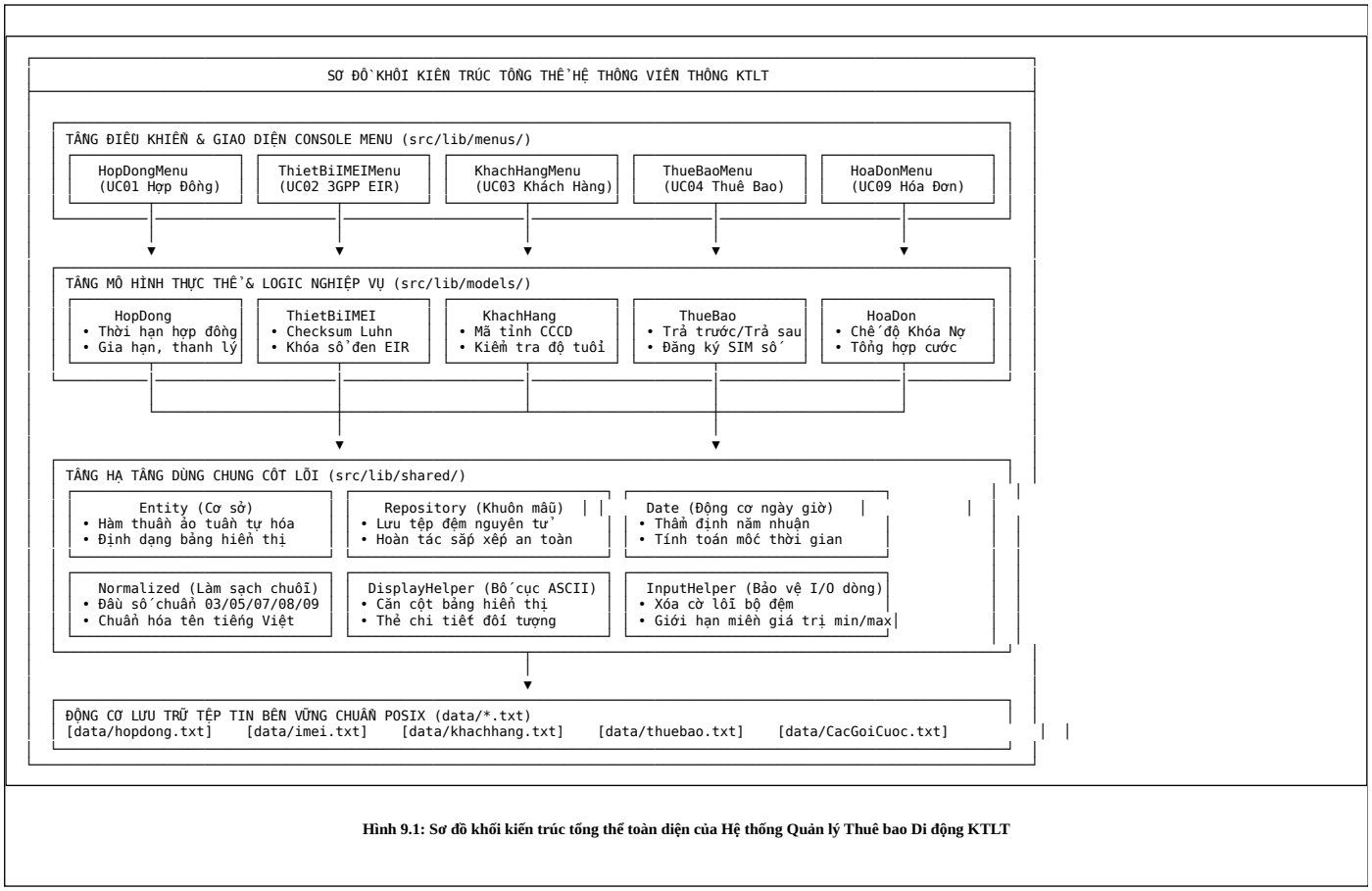
**Câu hỏi 3: Lớp `Repository<T>` bảo đảm an toàn dữ liệu và chống hỏng tệp khi mất nguồn đột ngột như thế nào?**

- **Trả lời**: `Repository<T>::saveToFile()` sử dụng cơ chế ghi đệm nguyên tử qua vùng nhớ RAM tmpfs ( `/dev/shm/ktlt_tmps` ). Dữ liệu được ghi ra một tệp tạm `.<ten_tep>.tmp` trên RAM. Chỉ sau khi tệp tạm được ghi trọn vẹn và đẩy sạch bộ đệm ( `flush` ), hệ thống mới thực thi lệnh `rename()` nguyên tử của hệ điều hành POSIX để thay thế tệp đích trên đĩa SSD. Nếu có sự cố mất nguồn hoặc ngoại lệ xảy ra trong lúc ghi, tệp tạm bị hủy bỏ, tệp gốc trên đĩa vẫn giữ nguyên trạng thái vẹn toàn 100%.
- **Dẫn chứng mã nguồn**: Tệp `src/lib/shared/Repository.h`, dòng 112–148.

---

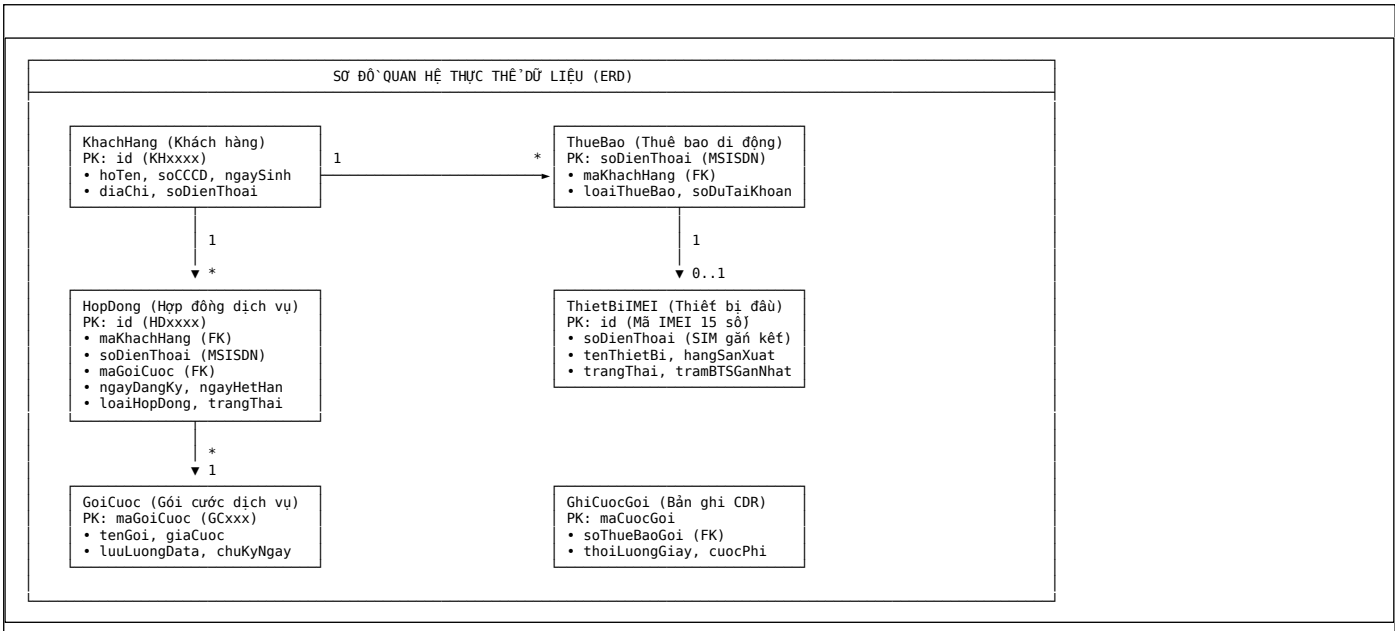
# 9. Sơ Đồ Hệ Thống & Lưu Đồ Khối Chi Tiết

## 9.1 Sơ Đồ Khối Kiến Trúc Toàn Diện Hệ Thống Viễn Thông




**Hình 9.1: Sơ đồ khối kiến trúc tổng thể toàn diện của Hệ thống Quản lý Thuê bao Di động KTLT**

## 9.2 Sơ Đồ Quan Hệ Thực Thể (ERD) & Ràng Buộc Khóa Ngoại




**Hình 9.2: Sơ đồ mối quan hệ giữa các thực thể và ràng buộc toàn vẹn khóa ngoại trong cơ sở dữ liệu**

# 10. Thuật Ngữ Viễn Thông & Tài Liệu Tham Khảo

## 10.1 Bảng Giải Thích Thuật Ngữ Viễn Thông & Công Nghệ Phần Mềm

| Thuật Ngữ | Tên Đầy Đủ Tiếng Anh | Ý Nghĩa Chuyên Ngành Viễn Thông / Phần Mềm |
|---|---|---|
| **3GPP** | 3rd Generation Partnership Project | Tổ chức hợp tác quốc tế xây dựng các tiêu chuẩn kỹ thuật cho mạng di động viễn thông (GSM, LTE, 5G). |
| **IMEI** | International Mobile Station Equipment Identity | Dãy 15 chữ số định danh phần cứng duy nhất cho từng máy điện thoại trên toàn cầu. |
| **TAC** | Type Allocation Code | 8 chữ số đầu của IMEI quy định hãng sản xuất và mẫu model thiết bị. |
| **SNR** | Serial Number | 6 chữ số tiếp theo đại diện cho số sê-ri xuất xưởng của thiết bị. |
| **CD** | Check Digit | Chữ số thứ 15 tính toán theo thuật toán Luhn Mod-10 để kiểm tra tính toàn vẹn. |
| **EIR** | Equipment Identity Register | Cơ sở dữ liệu của nhà mạng quản lý danh sách thiết bị Trắng, Xám và Đen. |
| **MSISDN** | Mobile Station International Subscriber Directory Number | Số điện thoại thuê bao di động sử dụng để gọi và nhận cuộc gọi. |
| **IMSI** | International Mobile Subscriber Identity | Mã số định danh thuê bao quốc tế được ghi cố định trong thẻ SIM. |
| **BTS** | Base Transceiver Station | Trạm thu phát sóng di động cố định quản lý kết nối vô tuyến với thiết bị đầu cuối. |
| **CDR** | Call Detail Record | Bản ghi chi tiết lưu vết thời gian bắt đầu, thời lượng và cước phí của mỗi cuộc gọi. |
| **BSS** | Business Support Systems | Hệ thống phần mềm quản lý hoạt động kinh doanh, khách hàng, hợp đồng và cước phí. |
| **OSS** | Operations Support Systems | Hệ thống phần mềm quản trị hạ tầng mạng vô tuyến, phần cứng và giám sát lưu lượng. |
| **eKYC** | Electronic Know Your Customer | Quy trình định danh và xác thực danh tính điện tử của khách hàng từ xa qua CCCD. |
| **CCCD** | Căn cước công dân | Mã số định danh 12 chữ số theo quy chuẩn quốc gia của Nước CHXHCN Việt Nam. |
| **POSIX** | Portable Operating System Interface | Bộ tiêu chuẩn quốc tế IEEE về giao tiếp hệ điều hành chuẩn Unix/Linux. |
| **RAII** | Resource Acquisition Is Initialization | Mẫu hình C++ gắn kết vòng đời tài nguyên vào tầm vực tồn tại của đối tượng. |
| **ASan** | AddressSanitizer | Trình quét bộ nhớ động phát hiện các lỗi truy cập vượt biên và rò rỉ bộ nhớ. |
| **UBSan** | UndefinedBehaviorSanitizer | Trình quét phát hiện các lỗi hành vi bất định trong thời gian chạy chương trình. |

## 10.2 Văn Bản Quy Phạm Pháp Luật & Tiêu Chuẩn Quốc Tế


1. **3GPP TS 22.016**: *Service Aspects: Mobile Station (MS) Features* — Tiêu chuẩn kỹ thuật viễn thông về tính năng thiết bị đầu cuối di động.
2. **3GPP TS 23.003**: *Technical Specification Group Core Network and Terminals: Numbering, Addressing and Identification* — Tiêu chuẩn quốc tế về cấu trúc IMEI, IMSI và MSISDN.
3. **GSM 02.16 (ETSI TS 100 908)**: *Digital Cellular Telecommunications System: International Mobile Station Equipment Identity (IMEI)* — Yêu cầu bảo mật định danh thiết bị viễn thông.
4. **Thông tư 22/2014/TT-BTTTT**: *Quy hoạch kho số viễn thông quốc gia* — Bộ Thông tin và Truyền thông ban hành.
5. **Nghị định 137/2015/NĐ-CP**: *Hướng dẫn chi tiết thi hành một số điều của Luật Căn cước công dân* — Chính phủ Nước CHXHCN Việt Nam ban hành.
6. **Đề bài Bài tập lớn Học phần Kỹ thuật Lập trình**: *Xây dựng ứng dụng quản lý bằng C++ theo hướng đối tượng, lưu trữ dữ liệu trên file* — Bộ môn Tin học, Khoa Viễn thông 1, PTIT.